

360° Blind-Spot Detection and Alert System

1. Title

360° Blind-Spot Detection and Alert System for Construction Vehicles Using Arduino and Ultrasonic Sensors

2. Team Members

- M BHAVANA
- MUMMADI REDDY RISHIKA
- VENKAT KARTHIKEYA KOTAKONDA
- RANJITH KUMAR C

3. Abstract

Construction vehicles such as excavators and loaders often have large blind spots that make it difficult for operators to notice nearby workers or obstacles. This can lead to serious accidents and safety risks at construction sites. Our project proposes a low-cost 360° blind-spot detection and alert system using ultrasonic sensors and Arduino.

The system continuously monitors the surroundings of the vehicle and detects nearby objects in real time. When an obstacle enters the danger zone, the system alerts the operator using LEDs and a buzzer. The project improves worker safety, reduces accidents, and increases operator awareness. The proposed system is affordable, simple to install, and suitable for construction environments.

4. Introduction

Heavy construction vehicles operate in crowded and dangerous environments where workers and obstacles may move close to the machine. Due to the large size of these vehicles, operators cannot clearly see all surrounding areas, creating blind spots. These blind spots are one of the major reasons for accidents at construction sites.

The proposed 360° blind-spot detection system uses ultrasonic sensors mounted around the vehicle to detect nearby objects. The sensors send distance data to the Arduino controller, which processes the information and activates warning indicators when an object comes too close.

5. Problem Statement

Construction vehicles such as excavators and loaders have large blind spots that prevent operators from seeing nearby workers, tools, or obstacles. This can result in accidents, injuries, equipment damage, and loss of life.

6. Objectives

- To design a 360° blind-spot monitoring system.
- To detect nearby objects using ultrasonic sensors.
- To provide audio and visual alerts to the operator.

- To improve worker and vehicle safety.
- To reduce accidents at construction sites.

7. Proposed Solution

The proposed system uses multiple ultrasonic sensors placed around the vehicle to monitor all directions. These sensors continuously measure the distance between the vehicle and nearby objects.

The Arduino microcontroller processes the sensor readings. If any object is detected within a predefined safety range, the system activates LEDs and a buzzer to alert the operator.

This real-time monitoring system helps the operator identify dangers quickly and avoid accidents.

8. Components / Technologies Used

Hardware Components

- Arduino Uno R3
- Ultrasonic Sensors (HC-SR04)
- LED (Neo pixel)
- Buzzer
- Breadboard
- Jumper Wires
- Power Supply

Software Used

- Arduino IDE-C++
- Processing-Java

Technologies

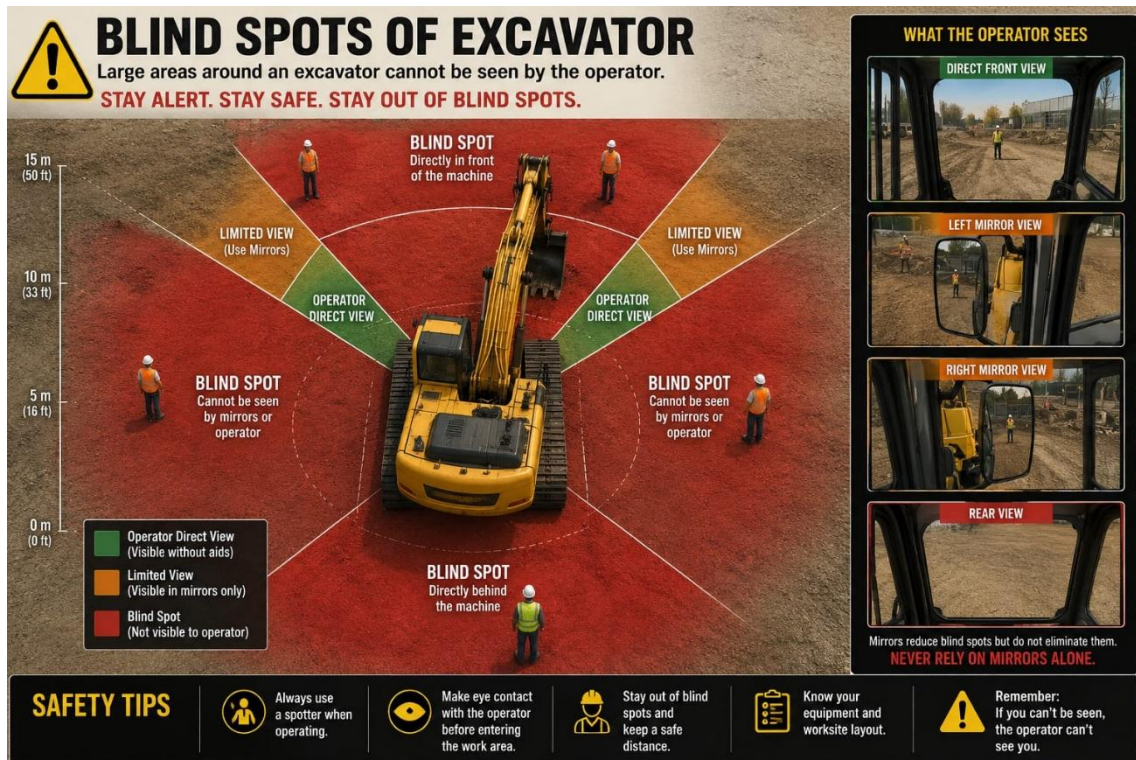
- IoT
- Embedded Systems
- Sensor-Based Monitoring

COST:

- Arduino Uno: ₹500 – ₹800
- Ultrasonic Sensors (HC-SR04 x4): ₹200 – ₹400
- LEDs and Buzzer: ₹100 – ₹200
- Breadboard and Jumper Wires: ₹100 – ₹200
- Power Supply / Battery: ₹200 – ₹500

Total Estimated Cost: ₹1100 – ₹2100 (Prototype Level)

Blind spot



9. Working Principle

1. Ultrasonic sensors are mounted around the vehicle.
2. The sensors continuously send ultrasonic waves.
3. When the waves hit an object, they reflect back to the sensor.
4. The Arduino calculates the distance using the reflected signal.
5. If the detected object is within the danger range, LEDs glow and the buzzer sounds.
6. The operator receives an immediate warning and can take action.

10. Architecture / Block Diagram

Ultrasonic Sensors → Arduino Uno → LED Indicators + Buzzer

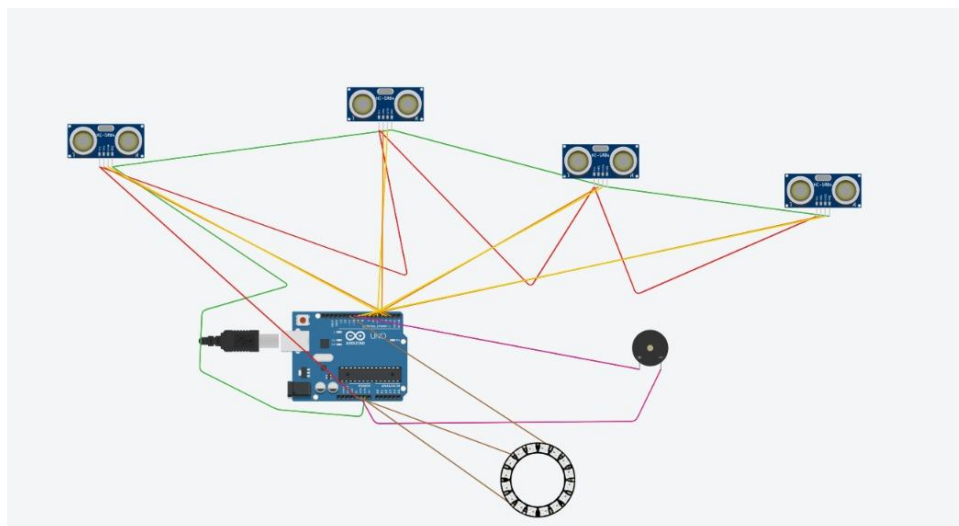
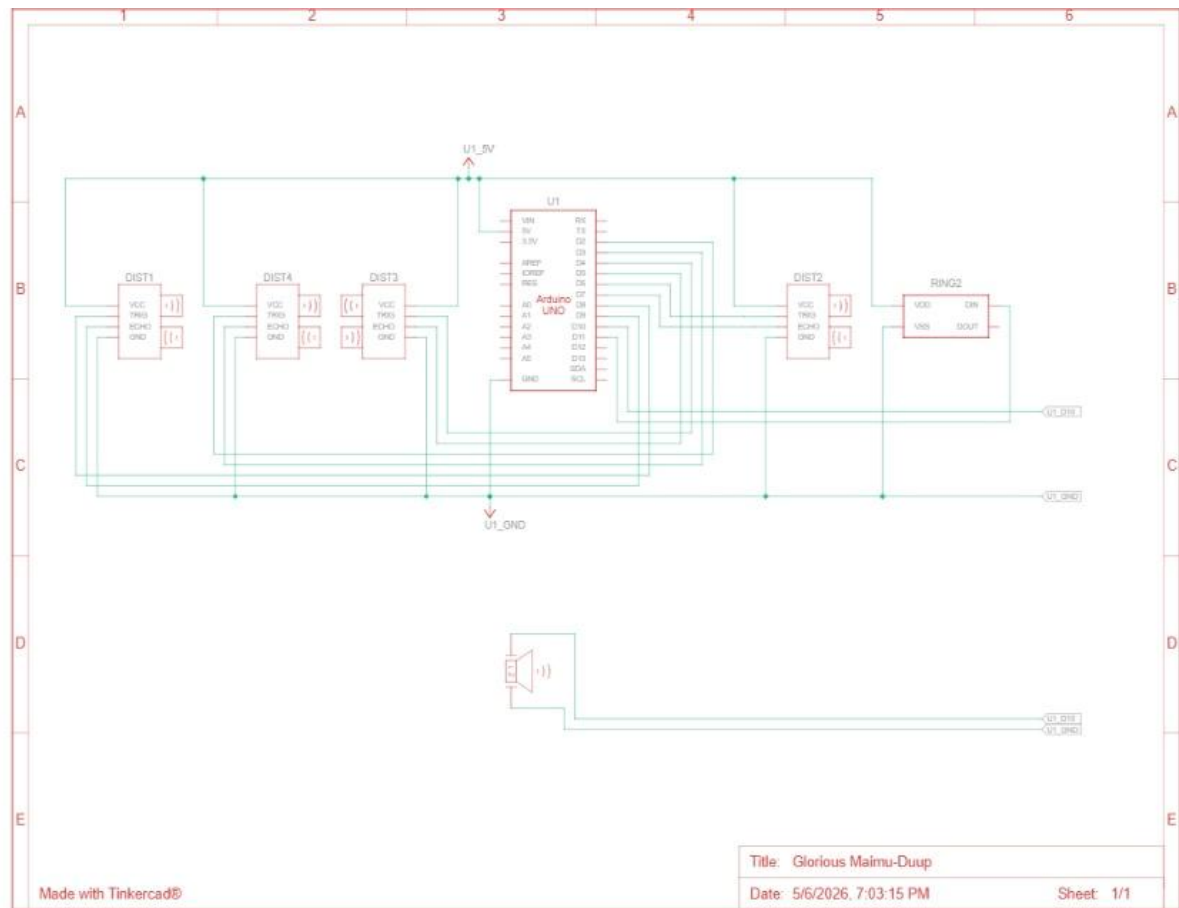
11. Implementation

The prototype was developed using Arduino Uno and ultrasonic sensors. Multiple sensors were connected around the model vehicle setup to achieve 360° coverage.

The Arduino program was written using Arduino IDE. The sensors continuously measured distance values and transmitted the data to the microcontroller. When the distance became smaller than the safety limit, the Arduino activated the buzzer and LEDs.

The system was tested using different obstacle positions and distances to verify detection accuracy.

CIRCUIT:



CODE:

EXCAVATOR CODE:

```
// =====  
// 360° EXCAVATOR BLIND-SPOT DETECTION SYSTEM  
//  
// Arduino UNO
```

```

// 4 HC-SR04 Sensors

// 16 LED NeoPixel Ring

// Dynamic Smart Buzzer

//

// RANGE SETTINGS:

// 0-6 cm -> RED ALERT

// 6-12 cm -> YELLOW ALERT

// 12-17 cm -> LOW WARNING

// >17 cm -> NO DETECTION

// =====

#include <Adafruit_NeoPixel.h>

// =====

// SENSOR PINS

// =====

// FRONT SENSOR

#define trigFront 2

#define echoFront 3

// BACK SENSOR

#define trigBack 4

#define echoBack 5

// LEFT SENSOR

#define trigLeft 6

#define echoLeft 7

// RIGHT SENSOR

#define trigRight 8

#define echoRight 9

// =====

// OUTPUT DEVICES

// =====

// BUZZER

#define buzzerPin 10

// 16 LED RING

```

```

#define ledPin 11

#define NUM_LEDS 16

Adafruit_NeoPixel ring(NUM_LEDS, ledPin, NEO_GRB + NEO_KHZ800);

// =====

// VARIABLES

// =====

long duration;

int frontDistance;

int backDistance;

int leftDistance;

int rightDistance;

int nearestDistance;

// =====

// DISTANCE FUNCTION

// =====

int getDistance(int trigPin, int echoPin)
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH, 30000);
    // NO ECHO
    if (duration == 0)
    {
        return 999;
    }
    int distance = duration * 0.034 / 2;
    // FILTER INVALID VALUES
    if (distance <= 1 || distance > 17)
    {
        return 999;
    }
}

```

```

    }
    return distance;
}

// =====

// SETUP

// =====

void setup()
{
    Serial.begin(9600);

    // FRONT
    pinMode(trigFront, OUTPUT);
    pinMode(echoFront, INPUT);

    // BACK
    pinMode(trigBack, OUTPUT);
    pinMode(echoBack, INPUT);

    // LEFT
    pinMode(trigLeft, OUTPUT);
    pinMode(echoLeft, INPUT);

    // RIGHT
    pinMode(trigRight, OUTPUT);
    pinMode(echoRight, INPUT);

    // BUZZER
    pinMode(buzzerPin, OUTPUT);

    // LED RING
    ring.begin();
    ring.show();

    Serial.println("360 Blind Spot System Started");
}

// =====

// LOOP

// =====

void loop()
{

```

```

// =====
// READ SENSOR DISTANCES
// =====

frontDistance = getDistance(trigFront, echoFront);
delay(25);

backDistance = getDistance(trigBack, echoBack);
delay(25);

leftDistance = getDistance(trigLeft, echoLeft);
delay(25);

rightDistance = getDistance(trigRight, echoRight);
delay(25);

// =====
// FIND NEAREST OBJECT
// =====

nearestDistance = 999;

if (frontDistance < nearestDistance)
    nearestDistance = frontDistance;

if (backDistance < nearestDistance)
    nearestDistance = backDistance;

if (leftDistance < nearestDistance)
    nearestDistance = leftDistance;

if (rightDistance < nearestDistance)
    nearestDistance = rightDistance;

// =====
// SEND DATA TO LAPTOP
// =====

Serial.print(frontDistance);
Serial.print(",");
Serial.print(backDistance);
Serial.print(",");
Serial.print(leftDistance);
Serial.print(",");
Serial.println(rightDistance);

```



```

// =====
// CLEAR LED RING
// =====

ring.clear();

// =====

// FRONT LEDs (0-3)
// =====

if (frontDistance != 999)
{
    for (int i = 0; i <= 3; i++)
    {
        // RED ALERT
        if (frontDistance <= 6)
        {
            ring.setPixelColor(i, ring.Color(255, 0, 0));
        }

        // YELLOW ALERT
        else if (frontDistance <= 12)
        {
            ring.setPixelColor(i, ring.Color(255, 255, 0));
        }

        // LOW WARNING
        else if (frontDistance <= 17)
        {
            ring.setPixelColor(i, ring.Color(255, 140, 0));
        }
    }
}

// =====

// RIGHT LEDs (4-7)
// =====

if (rightDistance != 999)
{

```

```

for (int i = 4; i <= 7; i++)
{
    if (rightDistance <= 6)
    {
        ring.setPixelColor(i, ring.Color(255, 0, 0));
    }
    else if (rightDistance <= 12)
    {
        ring.setPixelColor(i, ring.Color(255, 255, 0));
    }
    else if (rightDistance <= 17)
    {
        ring.setPixelColor(i, ring.Color(255, 140, 0));
    }
}

// =====

// BACK LEDs (8-11)

// =====

if (backDistance != 999)
{
    for (int i = 8; i <= 11; i++)
    {
        if (backDistance <= 6)
        {
            ring.setPixelColor(i, ring.Color(255, 0, 0));
        }
        else if (backDistance <= 12)
        {
            ring.setPixelColor(i, ring.Color(255, 255, 0));
        }
        else if (backDistance <= 17)
        {

```

```

    ring.setPixelColor(i, ring.Color(255, 140, 0));
  }
}
}

// =====

// LEFT LEDs (12-15)
// =====

if (leftDistance != 999)
{
  for (int i = 12; i <= 15; i++)
  {
    if (leftDistance <= 6)
    {
      ring.setPixelColor(i, ring.Color(255, 0, 0));
    }
    else if (leftDistance <= 12)
    {
      ring.setPixelColor(i, ring.Color(255, 255, 0));
    }
    else if (leftDistance <= 17)
    {
      ring.setPixelColor(i, ring.Color(255, 140, 0));
    }
  }
}

// =====

// NO OBJECT DETECTED
// =====

if (nearestDistance > 17 || nearestDistance == 999)
{
  noTone(buzzerPin);
  ring.show();
  delay(20);
}

```

```

    return;
}

// =====

// DYNAMIC WARNING SYSTEM
// =====

int beepDelay = map(nearestDistance, 1, 17, 40, 700);
beepDelay = constrain(beepDelay, 40, 700);
int toneFrequency = map(nearestDistance, 1, 17, 3200, 700);
toneFrequency = constrain(toneFrequency, 700, 3200);

// =====

// CRITICAL WARNING
// =====

if (nearestDistance <= 6)
{
    tone(buzzerPin, 3200);
    delay(25);
    noTone(buzzerPin);
    delay(10);
}

// =====

// NORMAL WARNING
// =====

else
{
    tone(buzzerPin, toneFrequency);
    delay(beepDelay);
    noTone(buzzerPin);
    delay(beepDelay);
}

// =====

// UPDATE LED RING
// =====

ring.show();

```

```
    delay(20);  
}
```

IOT CODE:

```
import processing.serial.*;  
  
// =====  
  
// VISION GUARD 360  
// AI BLIND-SPOT SAFETY SYSTEM  
// =====  
  
// =====  
  
// SERIAL  
// =====  
  
Serial myPort;  
String data = "";  
  
// =====  
  
// SENSOR VALUES  
// =====  
  
float front = 20;  
float back  = 20;  
float left  = 20;  
float right = 20;  
  
// =====  
  
// RADAR SETTINGS  
// =====  
  
float MAX_RANGE = 20;  
float DANGER_DIST = 6;  
float WARNING_DIST = 14;  
  
// =====  
  
// IMAGE + FONT  
// =====  
  
PImage excavator;  
PFont monoFont;
```

```

// =====

// RADAR SWEEP

// =====

float sweepAngle = 0;

// =====

// SETUP

// =====

void setup()
{
    // PERFECT FIT FOR 1920x1080

    size(1500, 820);

    smooth();

    frameRate(60);

    excavator = loadImage("excavator.png");

    monoFont = createFont("Courier New", 16, true);

    textFont(monoFont);

    println(Serial.list());

    // CHANGE PORT NUMBER IF NEEDED

    myPort = new Serial(this, Serial.list()[0], 9600);

    myPort.bufferUntil('\n');
}

// =====

// DRAW

// =====

void draw()
{
    background(2, 8, 10);

    drawDangerFlash();

    pushMatrix();

    // MOVED SLIGHTLY UP

    translate(width/2, height/2 - 40);

    drawGrid();

    drawRadarRings();
}

```

```

drawRadarSweep();
drawZoneArc(front, -HALF_PI);
drawZoneArc(back, HALF_PI);
drawZoneArc(left, PI);
drawZoneArc(right, 0);
drawExcavator();

// =====

// TARGETS

// =====

drawTarget(front, "FRONT");
drawTarget(back, "BACK");
drawTarget(left, "LEFT");
drawTarget(right, "RIGHT");
drawTitle();
popMatrix();
drawStatusPanel();
}

// =====

// GRID

// =====

void drawGrid()
{
    stroke(0, 255, 120, 10);
    strokeWeight(1);
    int spacing = 45;
    for(int x = -900; x <= 900; x += spacing)
    {
        line(x, -900, x, 900);
    }
    for(int y = -900; y <= 900; y += spacing)
    {
        line(-900, y, 900, y);
    }
}

```

```

}

// =====

// RADAR RINGS

// =====

void drawRadarRings()
{
  noFill();
  strokeWeight(1);
  // PERFECT FIT
  int[] radii = {80, 160, 240, 320, 400};
  for(int i = 0; i < radii.length; i++)
  {
    stroke(0, 255, 120, 30);
    ellipse(0, 0, radii[i]*2, radii[i]*2);
    fill(0, 255, 120, 100);
    textSize(14);
    text(i*5 + " cm",
         radii[i] + 10,
         0);
    noFill();
  }
  stroke(0, 255, 120, 18);
  line(-400, 0, 400, 0);
  line(0, -400, 0, 400);
}

// =====

// RADAR SWEEP

// =====

void drawRadarSweep()
{
  pushMatrix();
  rotate(radians(sweepAngle));
  noStroke();

```



```

fill(0, 255, 120, 20);
arc(0,
    0,
    800,
    800,
    radians(-10),
    radians(10));
popMatrix();
sweepAngle += 2.5;
if(sweepAngle >= 360)
{
    sweepAngle = 0;
}
}
// =====
// ZONE ARC
// =====

void drawZoneArc(float dist, float angle)
{
    float arcSpan = radians(80);
    float r1 = 80;
    float r2 = 400;
    noStroke();
    fill(getZoneColor(dist, 40));
    beginShape();
    for(float a = angle - arcSpan/2;
        a <= angle + arcSpan/2;
        a += 0.03)
    {
        vertex(r1*cos(a),
            r1*sin(a));
    }
    for(float a = angle + arcSpan/2;

```

```

    a >= angle - arcSpan/2;
    a -= 0.03)
{
    vertex(r2*cos(a),
           r2*sin(a));
}
endShape(CLOSE);
}

// =====

// COLOR

// =====

color getZoneColor(float dist, int alpha)
{
    if(dist <= DANGER_DIST)
    {
        return color(255, 40, 40, alpha);
    }
    if(dist <= WARNING_DIST)
    {
        return color(255, 180, 0, alpha);
    }
    return color(0, 255, 120, 25);
}

// =====

// EXCAVATOR

// =====

void drawExcavator()
{
    imageMode(CENTER);
    if(excavator != null)
    {
        noStroke();
        fill(0, 255, 120, 18);
    }
}

```

```

    ellipse(0, 0, 130, 130);
    image(excavator, 0, 0, 140, 280);
}
}
// =====

// RADAR TARGET
// =====

void drawTarget(float dist, String side)
{
    if(dist > MAX_RANGE)
    {
        return;
    }
    // PERFECT POSITIONING
    float radarDist =
        map(dist,
            0,
            20,
            50,
            400);
    radarDist =
        constrain(radarDist,
            50,
            400);
    float x = 0;
    float y = 0;
    if(side.equals("FRONT"))
    {
        y = -radarDist;
    }
    else if(side.equals("BACK"))
    {
        y = radarDist;
    }
}

```

```

}
else if(side.equals("LEFT"))
{
    x = -radarDist;
}
else if(side.equals("RIGHT"))
{
    x = radarDist;
}

// =====

// TARGET COLOR

// =====

color radarColor;
if(dist <= DANGER_DIST)
{
    radarColor = color(255, 40, 40);
}
else if(dist <= WARNING_DIST)
{
    radarColor = color(255, 180, 0);
}
else
{
    radarColor = color(0, 255, 120);
}

// =====

// RADAR LINE

// =====

stroke(radarColor, 120);
strokeWeight(2);
line(0, 0, x, y);

// =====

```

```
// TARGET BOX
// =====

stroke(radarColor);
noFill();
rectMode(CENTER);
rect(x, y, 30, 30);
// =====

// TARGET DOT
// =====

noStroke();
fill(radarColor);
ellipse(x, y, 16, 16);
// =====

// BLINK
// =====

float blink = sin(frameCount * 0.18);
if(blink > 0)
{
    fill(255);
    ellipse(x, y, 5, 5);
}
// =====

// PULSE
// =====

noFill();
stroke(radarColor, 180);
strokeWeight(2);
float pulse =
    map(sin(frameCount * 0.08),
        -1,
        1,
        22,
        55);
```

```

ellipse(x, y, pulse, pulse);
stroke(radarColor, 90);
ellipse(x, y, pulse + 18, pulse + 18);

// =====

// DISTANCE TEXT

// =====

fill(255);
textAlign(CENTER);
textSize(14);
text(int(dist) + " cm",
    x,
    y - 24);

// =====

// SIDE LABEL

// =====

fill(radarColor);
textSize(11);
text(side,
    x,
    y + 28);
}

// =====

// TITLE + CLOCK + GPS

// =====

void drawTitle()
{
    // MAIN TITLE
    fill(0,255,120);
    textAlign(CENTER);
    textSize(42);
    text("VISION GUARD 360",
        0,
        -370);

```

```

// SUBTITLE
fill(120);
textSize(18);
text("AI BLIND-SPOT SAFETY SYSTEM",
    0,
    -338);

// LIVE TIME
fill(0,255,120);
textSize(20);
String currentTime =
    nf(hour(),2) + ":" +
    nf(minute(),2) + ":" +
    nf(second(),2);
text(currentTime,
    640,
    -370);

// GPS STATUS
fill(0,200,255);
textSize(18);
text("GPS LINKED",
    -640,
    -370);
}

// =====

// DANGER FLASH

// =====

void drawDangerFlash()
{
    boolean danger =
        front <= DANGER_DIST ||
        back <= DANGER_DIST ||
        left <= DANGER_DIST ||
        right <= DANGER_DIST;

```

```

if(danger)
{
    float blink = sin(frameCount * 0.25);
    int alpha =
        int(map(blink,
            -1,
            1,
            0,
            18));

    noStroke();
    fill(255, 0, 0, alpha);
    rect(0, 0, width, height);
}
}

// =====

// STATUS PANEL

// =====

void drawStatusPanel()
{
    int px = 25;
    // MOVED UP
    int py = height - 150;
    noStroke();
    fill(0, 20, 10, 220);
    rect(px, py, 340, 150, 10);
    fill(0, 255, 120);
    textSize(20);
    textAlign(LEFT);
    text("SENSOR STATUS",
        px + 15,
        py + 25);
    drawStatusRow("FRONT",
        front,

```



```

        px + 15,
        py + 55);
drawStatusRow("LEFT",
    left,
    px + 15,
    py + 85);
drawStatusRow("RIGHT",
    right,
    px + 15,
    py + 115);
drawStatusRow("BACK",
    back,
    px + 15,
    py + 145);
}

// =====
// STATUS ROW
// =====

void drawStatusRow(String label,
    float dist,
    float x,
    float y)
{
    fill(170);
    textSize(16);
    text(label, x, y);
    float barX = x + 90;
    float barW = 130;
    noStroke();
    fill(40);
    rect(barX, y - 12, barW, 8, 3);
    float pct =
        constrain(map(dist,

```

```

        0,
        MAX_RANGE,
        barW,
        0),
    0,
    barW);
fill(getZoneColor(dist, 255));
rect(barX, y - 12, pct, 8, 3);
fill(255);
textAlign(RIGHT);
if(dist <= MAX_RANGE)
{
    text(int(dist) + " cm",
        barX + 190,
        y);
}
else
{
    text("CLEAR",
        barX + 190,
        y);
}
textAlign(LEFT);
}

// =====

// SERIAL EVENT

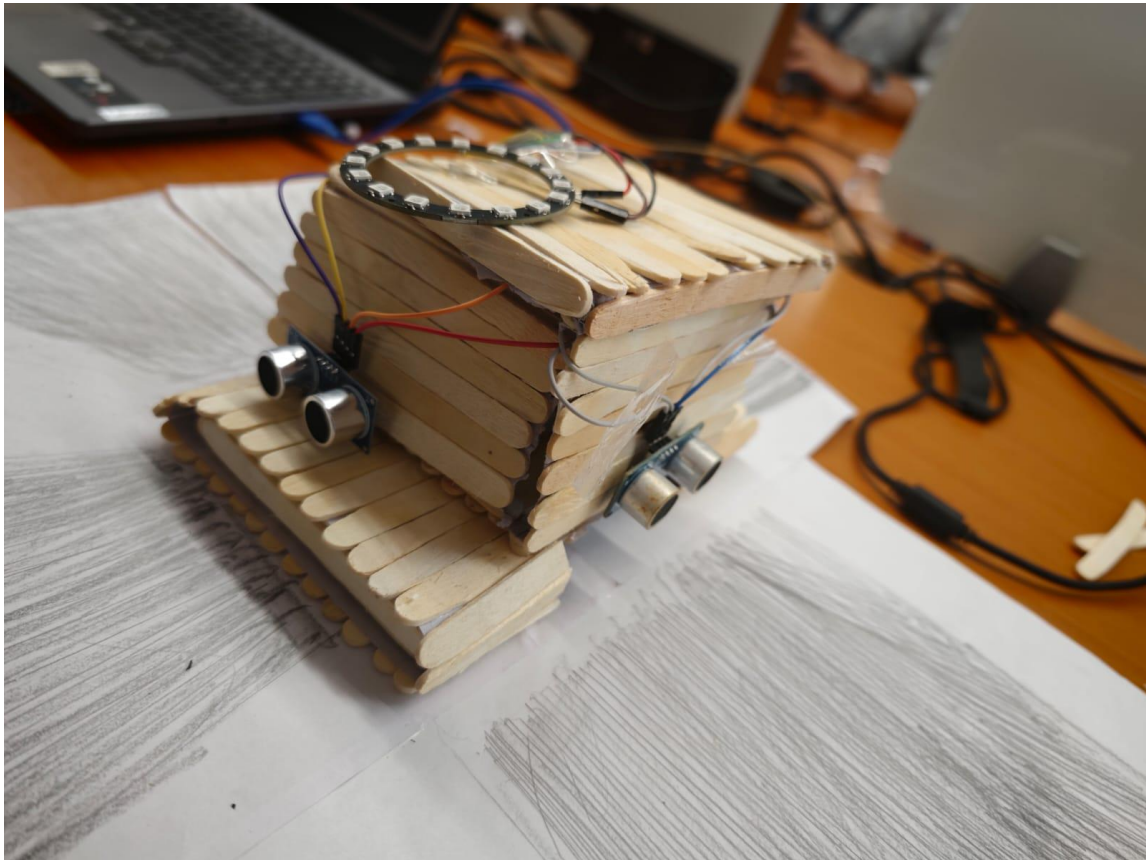
// =====

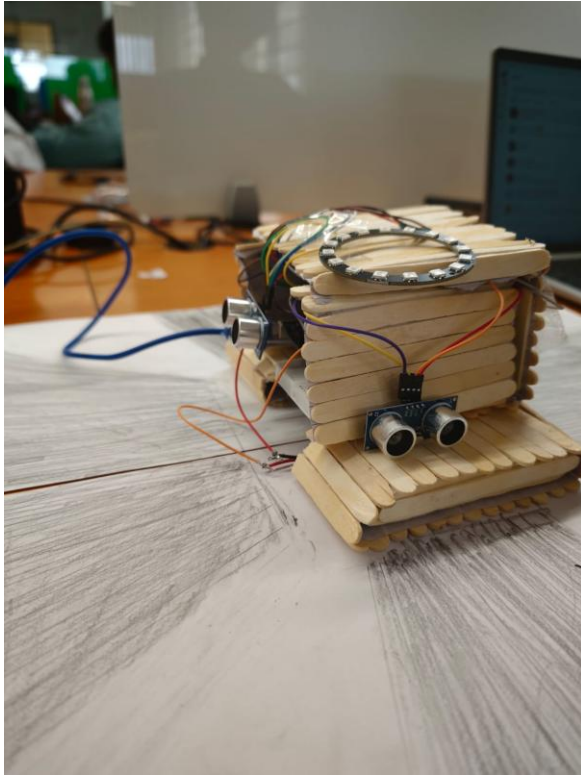
void serialEvent(Serial myPort)
{
    data = myPort.readStringUntil('\n');
    if(data != null)
    {
        data = trim(data);
    }
}

```

```
println(data);  
// FORMAT:  
// front,back,left,right  
String[] values = split(data, ',');  
if(values.length == 4)  
{  
    front = float(values[0]);  
    back  = float(values[1]);  
    left  = float(values[2]);  
    right = float(values[3]);  
}  
}  
}
```

Prototype:





12. Results / Output

- Successfully detected nearby obstacles.
- Provided real-time alerts using buzzer and LEDs.
- Achieved 360° monitoring around the prototype.
- Improved operator awareness and safety.
- Demonstrated low-cost and reliable operation.

13. Advantages

- Low-cost safety solution.
- Easy to install and maintain.
- Real-time obstacle detection.
- Improves construction site safety.
- Reduces accidents and collisions.
- Suitable for different heavy vehicles.

14. Applications

- Excavators
- Loaders
- Dump trucks
- Mining vehicles

- Industrial vehicles
- Warehouse machinery

15. Future Scope

- Integration with camera systems.
- Mobile app monitoring.
- GPS and IoT cloud connectivity.
- AI-based object recognition.
- Voice alert system.
- Automatic braking mechanism.

16. Conclusion

The 360° Blind-Spot Detection and Alert System is an effective solution for improving safety around construction vehicles. The project successfully detects nearby objects and alerts the operator in real time using sensors, LEDs, and a buzzer.

The system is affordable, reliable, and easy to implement, making it suitable for construction and industrial environments. This project demonstrates how IoT and embedded systems can help reduce accidents and improve workplace safety.

17. References

1. Arduino Official Documentation
2. HC-SR04 Ultrasonic Sensor Datasheet
3. IoT and Embedded Systems Study Materials
4. Construction Site Safety Research Papers
5. Arduino IDE Documentation
6. Chatgpt